

Mappa dei processi e stato di attuazione

Ogni pagina e ogni funzione di QUANTO, passo per passo — e confronto con i documenti iniziali: cosa c'è, com'è stato fatto, cosa manca o è diverso.

21 settembre 2026 · versione dell'app: commit 5924b1b · 298 test automatici superati

24

Fatto come da
progetto

13

Fatto in modo
diverso

24

Fatto in parte

18

Non fatto

1

Aggiunto, non
previsto

4

Fase futura nei
documenti

Sono 84 punti dei documenti iniziali confrontati con il codice (parte C). Le aggiunte «non previste» sono elencate a parte (parte E).

Come leggere questo documento

Il documento ha cinque parti. Prima si vedono i **processi** (cosa succede quando clicchi), poi i **motori** dietro le quinte, poi il **confronto con i documenti iniziali**, poi la lista di **cosa sistemare**.

- **Sintesi**
 - **Parte A** · Processi pagina per pagina
 - **Parte B** · I motori dietro le quinte
 - **Parte C** · Confronto con i documenti iniziali
 - **Parte D** · Cosa sistemare, in ordine di importanza
- **Parte E** · Cose aggiunte che non erano nei documenti
 - **Parte F** · Limiti di questa verifica
 - **Appendici** · tutte le funzioni dell'API, variabili di configurazione

I sei stati che trovi in tutto il documento

Fatto come da progetto

Il documento lo chiedeva e c'è, funziona come descritto.

Fatto in modo diverso

Funziona, ma non come progettato. Può essere una scelta o una sostituzione.

Fatto in parte

C'è una parte; un'altra parte del progetto manca.

Non fatto

Il documento lo chiedeva e non c'è. Da sistemare.

Aggiunto, non previsto

C'è, ma non era nei documenti iniziali (lo hai chiesto tu o l'ho aggiunto io).

Fase futura nei documenti

Il documento stesso lo rimanda a una fase successiva (Fasi 2, 3, 4).

I documenti iniziali che ho usato

Documento	Data	Come l'ho usato
FKOS_v2_completo.pdf	17 set 2026 · 21 pag.	RIFERIMENTO PRINCIPALE. Moduli 1-22 + schema API. È la versione più recente: qui l'ingestion della Fonte A è quella «a catalogo continuo, conferma del cliente, passaggi multipli».
FKOS_schema_processi_gio17SET2026.pdf	17 set 2026 · 10 pag.	RIFERIMENTO PRINCIPALE per i flussi: mappa generale, Fonti A/B/C, Missione Uno e Due, Demo, Console a 5 viste, go-to-market, fasi future.
DossierFKOS_v2.2 / v2.1 (16 set)	20 e 19 pag.	Versioni precedenti dello stesso documento. Differiscono da v2_completo solo sull'ingestion (lì c'era il «punteggio di confidenza» del modello al posto del confronto tra passaggi). Superate su quel punto.
Dossier_PROGETTOBUDGET (16 set)	19 pag.	Ancora più vecchio: contiene la Roadmap (Modulo 23, pilota ristretto a Transizione 5.0 con 10-15 criteri). Nelle versioni successive la roadmap è stata tolta.
raccontoBUDGET (15 set)	21 pag.	La prima visione: blockchain, «Local Edge Enclave», polizza assicurativa. Molte cose sono state riviste dopo (la blockchain l'hai tolta tu; l'enclave locale è diventata pseudonimizzazione sul server).
FKOS_piano_economico v2 / v3 (.xlsx)	6 fogli	Piano economico (ricavi, costi, conto economico, KPI). Non contiene processi da tradurre in codice: non l'ho confrontato riga per riga.

Regola che ho seguito: quando due documenti dicono cose diverse vale il più recente. Sull'ingestion delle regole (Fonte A) valgono quindi **FKOS_v2_completo** e lo **schema del 17 settembre** (catalogo continuo, conferma del cliente, passaggi multipli, audit di calibrazione), non i dossier v2.1/v2.2.

Sintesi

In breve. Il cuore del progetto è stato realizzato e funziona: il motore che calcola e valida un budget con 60 controlli esatti, la firma e la verifica dell'impronta, l'ottimizzatore dell'allocazione, il confronto tra letture di una regola con coda per il consulente, le esportazioni con QR. Quello che **manca** è quasi tutto ciò che sta **intorno ai dati**: le tabelle ufficiali, i documenti del cliente, la memoria che resta, i dati veri per l'allocazione e per il confronto, e l'intelligenza artificiale nei punti in cui i documenti la prevedono.

Stato per modulo dei documenti

Modulo	Sintesi	Punti
MODULO 1 · Missione	1 OK	1
MODULO 2 · Le due missioni	1 OK 1 DIV 2 PARZ 2 NO	6
MODULO 4-6 · Demo, banca dati pattern, pattern matching	2 OK 1 DIV 4 PARZ 4 NO	11
MODULO 7 · Disaccoppiamento motore / LLM (strict grounding)	3 OK 1 PARZ	4
MODULO 8 · Console di Budgeting	3 OK 1 DIV 4 PARZ	8
MODULO 9.1 · Fonte A: normativa e regole (versione finale)	2 OK 5 PARZ 3 NO 1 FUT	11
MODULO 9.2 · Fonte B: tabelle di mercato	1 PARZ 2 NO	3
MODULO 9.3 e 16 · Fonte C: dati del cliente e privacy	1 DIV 2 PARZ 1 NO	4
MODULO 10 · Da costo grezzo a riga validata	1 OK 1 DIV 1 PARZ	3
MODULO 11 · I 60 criteri	1 DIV 1 ADD	2
MODULO 12 · Allocazione annuale	4 OK 1 DIV 2 NO	7
MODULO 13 e 14 · Dove interviene l'AI · Metriche	3 OK 1 DIV	4
MODULO 15 · Stack e infrastruttura	1 OK 1 DIV 1 PARZ 2 NO	5
MODULO 17 · Registro crittografico (CEP)	3 OK 2 DIV 1 PARZ 1 NO	7
MODULO 18, 19 · Assicurazione e Reparto di consulenza	2 FUT	2
MODULO 20-22 · Go-to-market, prezzi, concorrenza	1 PARZ 1 NO 1 FUT	3
SCHEMA API · Endpoint e notifiche	2 DIV 1 PARZ	3

OK = come da progetto · DIV = diverso · PARZ = in parte · NO = non fatto · ADD = aggiunto · FUT = fase futura

Le cose più importanti da sistemare

1. **Memoria persistente.** Su Vercel il database (SQLite) si azzera: bandi cercati, documenti scaricati, esecuzioni e registro vanno persi a ogni riavvio del server.
2. **Tabelle ufficiali (Fonte B).** Le tabelle di CCNL, oneri, TFR, ammortamento e benchmark sono illustrative e coprono 7 combinazioni.
3. **Documenti del cliente (Fonte C).** Nessun parser di buste paga, bilanci, F24 e nessun OCR; nessun controllo di «confidenza di lettura».
4. **Missione Due con dati veri.** L'allocazione usa 3 spese e 3 fondi di esempio.
5. **Flusso di ricerca come da schema.** Manca la ricerca nell'elenco interno, la conferma «è il bando giusto?» e il controllo della cache.
6. **Accesso e sicurezza.** API aperta (autenticazione spenta), codice manager QUANTO_1, sito dietro la protezione Vercel.

Da sapere subito: finché mancano le tabelle ufficiali e la memoria persistente, QUANTO va presentato come **prototipo dimostrativo**: i numeri sono calcolati bene, ma le tabelle su cui si basano sono illustrative e tutto ciò che un utente carica si perde al riavvio del server.

Parte A · Processi pagina per pagina

Per ogni funzione trovi la catena di quello che succede: **cosa clicchi** → **cosa fa il browser** → **cosa fa il server** → **cosa vedi**. Sotto ogni processo c'è il confronto con il progetto. I nomi tra parentesi tonde (GET, POST...) sono le chiamate al server, utili a chi lavora sul codice.

1. Barra in alto e struttura generale

Visibile in ogni pagina

L'app è una pagina web (React) che parla con un server (FastAPI). Il browser non fa mai un calcolo economico: ogni cifra arriva dal server.

Apri l'app

- 1 Il browser carica la pagina
- 2 chiede al server l'elenco dei bandi (GET /bandi)
- 3 lo stato del registro (GET /registry/status)
- 4 il catalogo dei campi di una voce (GET /budget/fields)
- 5 i titoli dei 60 controlli (GET /budget/criteria)

Risultato: Compaiono le 7 pagine: Bandi, Budget, Allocazione, Confronto, Verifica, Guida, Quartier Generale.

Fatto in modo diverso I documenti prevedevano una barra laterale (Bandi Attivi, Bilancio Annuale, Allocazione Risorse, Demo Pattern, Report). Ho fatto una barra in alto; «Bilancio Annuale» non esiste e il «Report» è dentro il Budget.

Segnale del registro (pallino colorato)

- 1 Il server ricalcola tutta la catena di firme del registro
- 2 verde = integro
- 3 arancione = integro ma firmato con una chiave di prova
- 4 rosso = qualcuno ha alterato una voce (dice quale)
- 5 grigio = server non raggiungibile

Risultato: Chi guarda sa subito se le certificazioni sono affidabili.

Aggiunto, non previsto Non era nei documenti.

Algoritmo (vista grafica)

- 1 Non è nella barra
- 2 si apre solo dal pulsante «Guarda come ha lavorato» della pagina Budget
- 3 «Torna al Budget» per uscire

Risultato: Richiesta tua del 20 settembre.

Aggiunto, non previsto Non era nei documenti (il documento parla di «Ispettore di riga», non di un film dell'algoritmo).

2. Pagina «Bandi»

Prima pagina. Serve a trovare un bando, leggere le sue regole e sceglierlo.

Contiene l'elenco dei bandi in memoria, la scheda di ognuno e il pannello «Cerca un bando sul web».

Vedere l'elenco e aprire una scheda

- 1 Apri «Bandi»
- 2 GET /bandi: nome, ente, stato (APERTO / CHIUSO / PERMANENTE / SANDBOX / IN LAVORAZIONE), numero di regole, requisiti e controlli attivi su 60
- 3 clicchi un bando
- 4 GET /bandi/{id}
- 5 compare la scheda con 4 schede: Regole · Requisiti · Controlli attivati · Fonti

Risultato: I bandi con ricerca senza esito NON compaiono all'utente (li vede solo il manager nell'Archivio).

Fatto come da progetto Elenco e scheda come nel «Catalogo Bandi» dello schema.

Fatto in parte Non c'è una casella per cercare un bando per nome nell'elenco interno (la ricerca per nome esiste solo sul web, vedi sotto).

Scheda → Regole

- 1 Ogni regola mostra: nome in parole semplici, valore, affidabilità (PRIMARIA / SECONDARIA / INTERPRETAZIONE / PARSING), controlli collegati (#7, #31...), fonte con la frase o il documento
- 2 le regole in disaccordo e quelle «da decidere» NON si vedono all'utente (solo al manager)

Risultato: L'utente sa da dove viene ogni numero.

Fatto come da progetto Come il principio «ogni numero ha la sua origine» (Modulo 8.1) e «regola pubblicata con riferimento tracciato» (schema p.3).

Scheda → Requisiti

- 1 Elenco di obblighi, divieti, limiti e informazioni ricavati dai documenti
- 2 campo di ricerca nel testo
- 3 filtri per tema (agevolazione, importi, chi può presentare domanda, età, territori, scadenze...)
- 4 «Mostra altri 40»
- 5 i requisiti «da rivedere» sono nascosti all'utente

Risultato: Requisiti anche senza numeri, ciascuno collegato ai controlli che lo verificano.

Aggiunto, non previsto I documenti parlano di regole numeriche; l'elenco dei requisiti in prosa l'ho aggiunto io.

Scheda → Controlli attivati

- 1 Griglia dei 60 controlli
- 2 giallo = attivato da una regola del bando
- 3 azzurro = si fa solo sui dati della voce
- 4 grigio = spento perché il bando non dà la regola

Risultato: Dice quali dei 60 controlli valgono per quel bando.

Aggiunto, non previsto Trasparenza sulle lacune: nei documenti non c'era, coerente con «se manca un dato il controllo non parte».

Scheda → Fonti

- 1 Documenti scaricati (ufficiale / secondaria / caricato a mano) con link e impronta SHA-256
- 2 atti di legge citati (solo se hanno data o anno)
- 3 avviso: «fai controllare le regole a un consulente»

Risultato: La prova di dove vengono regole e requisiti.

Fatto come da progetto Rispetta «regola sempre tracciata al testo di origine».

Usa questo bando nel budget

- 1 Clicchi il pulsante
- 2 GET /bandi/{id} e POST /bandi/{id}/select (l'operazione entra nella timeline del manager)
- 3 le regole pubblicate diventano il «set di regole» del budget
- 4 si apre la pagina Budget
- 5 se il bando non ha regole pubblicate: errore «non ha ancora regole pubblicate»

Risultato: Il bando scelto compare nel chip «In uso».

Fatto come da progetto Corrisponde a «conferma bando → trigger» solo in parte: qui non c'è estrazione al clic (vedi sotto).

Cerca un bando sul web (ricerca, download, lettura)

- 1 Scrivi il nome (almeno 3 lettere), eventualmente parole in più e link ufficiali che già conosci
- 2 clicchi «Cerca e scarica»
- 3 POST /bandi/research/search: crea la scheda nel catalogo (solo nome) e cerca, in questo ordine: (1) mappa delle sigle di azione (KA152, KA153, KA154, KA210, KA220) → pagina della Guida del programma; (2) cataloghi nazionali incentivi.gov.it (5.889 misure) e Invitalia, con confronto anche approssimativo; (3) portali ufficiali per famiglia di programma, seguendo i link per 2 livelli; (4) motori di ricerca: Brave / Serper / Tavily se c'è la chiave, poi Bing e DuckDuckGo
- 4 ogni risultato riceve un punteggio di pertinenza (sul percorso dell'indirizzo, mai sul dominio) e la classe UFFICIALE o SECONDARIA
- 5 il browser costruisce una coda: prima i risultati ufficiali, poi PDF e atti di legge trovati dentro le pagine (max 16 documenti, profondità 2)
- 6 per ogni documento POST /bandi/research/fetch: controlli anti-SSRF (solo indirizzi pubblici, http/https, porte 80/443, redirect ricontrollati, limiti di peso e tempo, max 120 richieste ogni 10 minuti) → scarica → converte PDF / Word / pagina in testo → salva testo, file originale e impronta
- 7 a fine download POST /bandi/research/analyze: legge tutti i documenti (quelli molto lunghi solo nei passaggi che nominano il bando), estrae regole numeriche, requisiti, atti citati e, per ogni documento, il motivo se non ha reso nulla
- 8 a schermo: documenti scaricati, regole, requisiti, atti di legge

Risultato: Il bando resta in memoria con testo e file originali. Se la ricerca non trova nulla, il messaggio lo dice e puoi incollare un link.

Fatto in parte Schema: Stadio 1 = agente AI che cataloga in continuo tutte le fonti; qui la ricerca parte a richiesta e non usa AI. Il catalogo nazionale è tenuto in memoria 6 ore, non in database. La Gazzetta Ufficiale non è scansionata.

Non fatto Schema: dopo la ricerca il cliente deve «confermare che è il bando giusto» prima dell'estrazione, e il sistema controlla la cache. Qui scarica e legge subito, senza conferma né controllo della cache (i due passaggi esistono come funzioni del server, /ingestion/confirm, ma non sono collegati a questo pannello).

Aggiunto, non previsto Il fatto che scarichi tutto, salvi i file originali e spieghi zero-risultati per documento non era nei documenti.

Aggiungi un documento a mano

- 1 Sotto la ricerca: «Aggiungi un documento a mano»
- 2 scegli un PDF (o Word) o incolli un testo di almeno 100 caratteri
- 3 POST /bandi/upload
- 4 stessa analisi della ricerca

Risultato: Serve quando la ricerca non trova la fonte ufficiale.

Aggiunto, non previsto Non era nei documenti.

Non fatto PDF scansionati (solo immagini): non si leggono, manca l'OCR, e il sistema lo dichiara.

Quadri normativi di riferimento

- 1 In fondo alla colonna sinistra
- 2 GET /bandi/references
- 3 si vede per esempio il Regolamento UE 2023/2831 sul de minimis, con il dato letto e il limite «letto da sintesi, non dal testo EUR-Lex»

Risultato: Contesto per il controllo #49.

Aggiunto, non previsto Non era nei documenti.

3. Pagina «Budget» (Missione Uno)

Serve a inserire le spese del progetto e farle controllare da tutti i 60 controlli del bando scelto.

Il numero della riga più importante del progetto: tutto ciò che compare qui è calcolato dal server.

Scegliere bando e nome progetto

- 1 Scegli il bando nel menù (o arrivi da «Usa questo bando»)
- 2 stessa procedura di selezione (regole + timeline)
- 3 scrivi il nome del progetto (di default PRJ-2026-001)

Risultato: Compaiono: sigla del bando, «N regole · il bando attiva X dei 60 controlli».

Fatto come da progetto

Riempire il budget: 5 vie

- 1 «Prova completa (46 voci)» → GET /budget/demo?mode=stress: 46 voci, ognuna pensata per mettere alla prova un controllo preciso, con le date adattate alla finestra del bando
- 2 «Progetto realistico (15 voci)» → stesso servizio, modalità realistic
- 3 «Importa Excel/CSV» → POST /budget/import: legge le righe, se ci sono errori li dice riga per riga
- 4 «Template Excel» → GET /budget/template.xlsx: modello con colonne, esempi e guida ai campi
- 5 «Aggiungi voce» → scegli la categoria (Personale, Beni strumentali, Consulenze, Spese generali, Formazione) e compili a mano

Risultato: Elenco di voci pronte da controllare.

Fatto in parte Documenti: le voci arrivano dalla Fonte C (buste paga, F24, bilanci, piano dei conti, con OCR e parser). Qui: solo inserimento manuale, Excel/CSV o scenari di prova. Nessun caricamento di documenti contabili.

Modificare una voce

- 1 Clicchi la voce in elenco
- 2 scheda «Modifica voce»
- 3 campi raggruppati (personale, beni, consulenze, spese generali, sottotipi...) generati da GET /budget/fields: numeri, date, sì/no, liste
- 4 Duplica / Elimina
- 5 se il budget era già stato controllato, dopo 0,7 secondi il server lo ricontrolla da solo

Risultato: Le modifiche restano nella pagina; il calcolo lo rifà il server.

Fatto come da progetto Rispetta «ogni interazione = nuova chiamata API, mai calcolo nel frontend».

Controlla il budget

- 1 Clicchi «Controlla il budget»
- 2 POST /budget/validate con nome progetto, regole del bando e voci
- 3 il server esegue la pipeline (vedi Parte B): controlli di ogni voce → tempo dedicato per persona → limiti in % sul totale → controlli sull'intero budget → impronta di ogni voce → albero di Merkle → riassunto scritto verificato
- 4 risposta: punteggio, richiesto, ammesso, escluso, voci con esito, controlli di budget, riassunto, radice di Merkle, codice CEP, traccia completa

Risultato: Quattro riquadri (Punteggio /100, Richiesto, Ammesso, Escluso o ridotto) e l'elenco delle voci per categoria: Ammessa / Ridotta / Respinta / In attesa di documento.

Fatto come da progetto Missione Uno del Modulo 2.1: ammissibilità voce per voce, importo esatto, tabella validata. Valori di tabella (Fonte B) illustrativi: vedi Parte C.

Fatto in parte Non ripartisce le voci tra pacchetti di lavoro (WP): i tetti in % si applicano al totale per categoria.

Ispettore di riga

- 1 Clicchi una voce
- 2 scheda «Ispettore»: da dove vengono i dati (documento Fonte C, tabella Fonte B con versione, regole Fonte A con versione) · come si calcola (RAL, oneri, TFR, costo annuo, costo orario ÷ ore, tempo dedicato × durata) · controllo per controllo (superato / ridotto / respinto e perché) · controlli non valutati · impronta SHA-256 della voce

Risultato: Risponde a «perché questo numero e non un altro?».

Fatto come da progetto Modulo 8.1 e Vista 1 (Budget Canvas + Ispettore).

Fatto in parte Si clicca la voce, non ogni singolo numero.

Controlli sull'intero budget

- 1 Riquadro sotto l'elenco: #48 cumulo con crediti d'imposta (intensità di aiuto), #49 de minimis, #55 cash-flow, #56 variazione tra capitoli, #60 coerenza globale
- 2 se un dato manca il controllo risulta «non valutato»

Risultato: Il punteggio è la percentuale (per difetto) di controlli superati su quelli realmente eseguiti.

Fatto come da progetto Conformity Score del Modulo 14.1.

Guarda come ha lavorato

- 1 Clicchi il pulsante
- 2 si apre la pagina Algoritmo con la traccia reale (non una simulazione)

Risultato: Vedi la sezione «Algoritmo».

Aggiunto, non previsto

Esporta Excel / PDF

- 1 Clicchi «Excel» o «PDF»
- 2 POST /budget/export/xlsx (o pdf)
- 3 file con tabella validata, controlli di budget, codice CEP-ID e QR verso la pagina Verifica

Risultato: Layer 1 del «dual-layer output» (per la PA) + Layer 2 (QR per i revisori).

Fatto come da progetto Excel e PDF con CEP-ID e QR come nel Modulo 17.3.

Fatto in parte Vista 4 «Report» prevede anche le note esplicative dell'LLM dentro il file e l'export del piano di allocazione: i file oggi non contengono il riassunto scritto e l'Allocazione non ha export.

Registra l'impronta

- 1 Clicchi «Registra l'impronta»
- 2 si apre una finestra con codice CEP, progetto (in forma opaca) e Merkle Root
- 3 «Registra» → POST /registry/register
- 4 il server aggiunge una voce al registro: progetto opaco + radice + istante + hash della voce precedente + firma Ed25519
- 5 ricevi l'attestazione (numero, istante, hash, firma, chiave)
- 6 un secondo tentativo sullo stesso progetto è rifiutato

Risultato: Prova che quell'impronta esisteva a quella data e non è stata cambiata.

Fatto in modo diverso I documenti (Modulo 17) chiamano «registro crittografico» l'insieme hash + Merkle + Auditor Portal; il racconto iniziale parlava di ancoraggio su blockchain, che hai fatto togliere. Al suo posto: catena di hash firmata dal server.

Fatto in modo diverso I documenti mettono il CEP in Fase 2 (dopo il pilota). L'ho costruito subito.

4. Vista «Algoritmo» (solo da Budget)

Mostra come il motore ha lavorato, passo per passo.

Non ricalcola nulla: legge la traccia che il server ha già prodotto.

Riprodurre la traccia

- 1 Clicchi «Riproduci» (oppure trascini la barra, «Avanti veloce», scegli velocità Lento / Normale / Veloce)
- 2 i controlli compaiono uno alla volta nell'ordine reale (fino a 540 passi con la prova completa)
- 3 i contatori sopra contano superati / ridotti / respinti / in attesa di documento

Risultato: Vedi l'ordine vero dell'algoritmo.

Aggiunto, non previsto Non era nei documenti.

Le 7 fasi

- 1 Riga di schede: Controlli su ogni voce · Tempo per persona · Limiti in % · Controlli sull'intero budget · Impronta di ogni voce · Albero di Merkle · Riassunto
- 2 ogni scheda dice quante operazioni ha fatto e quanti millisecondi

Risultato: Rende visibile il «flusso» dello schema p.5.

Aggiunto, non previsto

Mappa dei 60 controlli

- 1 Griglia righe = voci, colonne = 60 controlli
- 2 verde superato · arancio ridotto · rosso respinto · azzurro in attesa di documento · tratteggiato non valutato · grigio non pertinente
- 3 clicchi una riga

Risultato: Vedi subito quali controlli hanno lavorato su quale voce.

Aggiunto, non previsto

Come scende l'importo

- 1 Dopo il clic su una voce: cascata «richiesto → controlli che tagliano → ammesso» con l'importo tolto da ciascun controllo

Risultato: Spiega ogni riduzione.

Aggiunto, non previsto

Limiti in % sul totale

- 1 Barre richiesto / ammesso per consulenze (#31), spese generali (#36), comunicazione (#37), beni immateriali (#26)
- 2 nota: il limite è sul totale FINALE, che dipende a sua volta dalle riduzioni; il motore risolve il circolo con formula esatta, senza tentativi

Risultato: Mostra i massimali applicati.

Fatto come da progetto Criteri 31 e 36 del Modulo 11.

Albero di Merkle

- 1 In fondo: l'albero con una foglia per ogni voce, i nodi intermedi e la radice
- 2 sotto: radice e codice CEP
- 3 link alla spiegazione passo passo

Risultato: Collega la voce alla radice certificabile.

Fatto come da progetto Modulo 17.2.

5. Pagina «Allocazione» (Missione Due)

Pianifica l'anno: quale fondo paga quale spesa e quanto resta a carico dell'ente.

Usa un ottimizzatore matematico esatto (programmazione lineare intera, HiGHS via scipy).

Aprire la pagina

- 1 Apri «Allocazione»
- 2 POST /allocation/optimize con i dati DI ESEMPIO fissi nel codice: 3 spese storiche (personale 150.000 €, beni 120.000 € ad aprile, consulenze 40.000 €) e 3 fondi (Transizione 5.0, FSE+ Occupazione, contributo locale de minimis con plafond 60.000 €)
- 3 il risolutore cerca la combinazione che rispetta tetti dei fondi, quote massime per categoria, fondi non cumulabili, plafond de minimis e finestre mensili

Risultato: Quattro riquadri (spesa totale, coperta dai fondi, a carico dell'ente, quota coperta), flusso «da dove arrivano i soldi», piano voce per voce, uso dei fondi con margine, grafico mese per mese, dettagli tecnici del risolutore.

Fatto in parte Il motore è quello del Modulo 12. Ma i dati sono un esempio: manca il passo 1 (caricare il bilancio storico dell'ente e categorizzarlo da solo) e il passo 2 (cercare nella Fonte A i fondi attivi compatibili). Oggi i fondi sono scritti nel codice.

Cambiare obiettivo

- 1 Menù «Obiettivo»: pagare il meno possibile di tasca propria · coprire più voci · usare meno fonti
- 2 nuova chiamata al server con la nuova funzione obiettivo

Risultato: Piano ricalcolato.

Fatto come da progetto Le 3 funzioni obiettivo del Modulo 12.2 e dello schema p.6.

Provare a togliere un fondo (what-if)

- 1 Clicchi il pulsante di un fondo
- 2 il fondo entra in «excluded_funds»
- 3 nuova chiamata: il risolutore rifà tutto senza quel fondo

Risultato: Vedi quanto sale la spesa a carico dell'ente.

Fatto come da progetto Passo 6 del Modulo 12.3.

Non fatto Nella Vista 2 del documento si può anche cliccare un mese per vedere le voci e «spostare questa voce dal Bando X al Fondo Y»: qui non c'è il clic sul mese né lo spostamento della singola voce.

Riepilogo scritto

- 1 Il server compone una frase di riepilogo con le cifre del piano

Risultato: Testo in fondo alla pagina.

Fatto in modo diverso Il documento prevede che lo scriva un LLM sotto controllo; qui è un modello fisso (nessun LLM collegato).

6. Pagina «Confronto» (Demo pattern)

Mostra a quale «tipo» di budget vincente somiglia il tuo.

Confronta come si ripartisce la spesa, non il testo del progetto.

Confrontare le quote

- 1 Compili 4 quote da 0 a 1 (personale, beni strumentali, consulenze, spese generali); già precompilate 0,58 / 0,12 / 0,25 / 0,05
- 2 clicchi «Confronta»
- 3 POST /pattern/match con categoria fissa «TRANSIZIONE_5.0»
- 4 il server porta le quote a somma 1, calcola la somiglianza del coseno con 4 archetipi scritti a mano, sceglie il più vicino (a parità, il nome che viene prima)
- 5 calcola lo scostamento maggiore in punti percentuali

Risultato: Archetipo più vicino, somiglianza, medie dell'archetipo, scostamento principale, frase di raccomandazione.

Fatto come da progetto Pattern Similarity Score con distanza del coseno (Modulo 6.3, 14.2). Nessun LLM nel calcolo (6.4).

Non fatto Non c'è la scelta del bando di riferimento (passo 1 della demo): la categoria è fissa e non cambia il risultato.

Non fatto Banca dati vera (graduatorie pubbliche, dati UE, dati del pilota) e clustering k-means: gli archetipi sono 4 esempi illustrativi scritti nel codice.

Fatto in parte Manca il recupero dei 5-10 budget più simili, la sesta categoria (formazione/comunicazione), il caricamento del budget dalla pagina Budget e la simulazione «sposta gli importi» in tempo reale: qui si cambiano le quote e si preme Confronta.

Fatto in modo diverso Il Conformity Score in parallelo (passo 6 della demo) c'è ma nella pagina Budget, non in questa.

7. Pagina «Verifica» (Auditor Portal)

Per chi deve fidarsi: controlla che un budget certificato non sia stato modificato.

Si arriva anche dal QR stampato sui file esportati (il QR precompila progetto e radice).

Verifica dell'impronta

- 1 Scrivi progetto e Merkle Root (0x...)
- 2 clicchi «Verifica impronta»
- 3 GET /registry/verify/{progetto}?merkle_root=...
- 4 il server controlla: il progetto è registrato? la radice coincide? la firma della voce è valida? l'INTERA catena è integra?
- 5 verdetto verde «certificato e non modificato», arancione «nessuna registrazione trovata» o rosso «verifica non superata»

Risultato: Un progetto non registrato non risulta mai valido.

Fatto come da progetto Auditor Portal del Modulo 17.3.

Ricalcola dai dati originali

- 1 Clicchi «Ricalcola dai dati originali»
- 2 POST /registry/verify/recompute con regole e voci del progetto
- 3 il server ricalcola la radice con lo stesso motore e la confronta col registro

Risultato: Prova indipendente: se cambia un euro, la radice cambia.

Fatto come da progetto «Ricalcolo istantaneo dell'Albero di Merkle» dello schema p.10.

Simula una manomissione

- 1 Spunta «Simula una manomissione (+1 € sulla prima riga)»
- 2 ripeti il ricalcolo
- 3 risultato negativo, per far vedere come funziona

Risultato: Didattico.

Aggiunto, non previsto

Verifica offline

- 1 GET /registry/public-key dà la chiave pubblica Ed25519
- 2 POST /registry/verify/attestation controlla un'attestazione con la sola chiave pubblica, senza accedere al registro

Risultato: Chi non si fida del server può controllare da solo.

Aggiunto, non previsto Non era nei documenti.

Report di asseverazione

- 1 Il portale mostra il verdetto e i dettagli a schermo

Risultato: —

Fatto in parte Il documento prevede un «Report di Asseverazione Crittografica» prodotto in 45 secondi: manca un documento scaricabile, e i 45 secondi non sono mai stati misurati.

8. Pagina «Guida»

Spiegazioni con parole semplici ed esempi.

Quattro sezioni; in tutte le altre pagine ci sono i pallini «?» e le parole sottolineate a puntini che aprono una spiegazione.

Percorso rapido

- 1 5 passi: scegli il bando → carica il budget → controlla → capisci perché → certifica e verifica
- 2 ogni passo è un pulsante che porta alla pagina giusta

Aggiunto, non previsto

Merkle Root passo passo

- 1 8 passi con un albero d'esempio: cosa è un'impronta, come si uniscono a coppie, la radice, la prova di una riga, «e se cambio 1 €?»
- 2 usa POST /registry/merkle-lab (stesso algoritmo del sistema)

Risultato: Spiegazione richiesta il 20 settembre.

Aggiunto, non previsto

Ogni funzione, con un esempio

- 1 Elenco di ogni pagina e funzione con cosa fa e un esempio con cifre inventate

Aggiunto, non previsto

Glossario

- 1 Oltre 40 termini spiegati (bando, RAL, FTE, TFR, de minimis, DNSH, hash, Merkle...)

Aggiunto, non previsto

9. Quartier Generale (area del manager)

Accesso con codice. Vede e può modificare tutto.

Il codice di partenza è QUANTO_1: va cambiato e non va dato agli utenti.

Entrare

- 1 Scrivi il codice → POST /hq/login
- 2 il server lo confronta; dopo 5 tentativi sbagliati blocca per 10 minuti
- 3 ti dà un gettone che il browser tiene solo per la sessione (X-HQ-Token)
- 4 ogni accesso e ogni tentativo finiscono nella timeline

Risultato: Si aprono le 8 sezioni.

Fatto come da progetto Schema p.8: «sessione solo team (non cliente)».

Fatto in parte Il codice predefinito è debole (in produzione è ancora QUANTO_1): va cambiato con QUANTO_HQ_CODE.

Panoramica

- 1 GET /hq/overview
- 2 contatori: budget controllati, progetti, bandi, documenti, eventi, registrazioni
- 3 memoria: tipo, peso, avviso «su hosting serverless la memoria si azzerà»
- 4 sicurezza: codice, firma del registro
- 5 attività degli ultimi giorni e ultimi eventi

Aggiunto, non previsto

Timeline

- 1 GET /hq/timeline (filtrabile per operazione)
- 2 ogni riga: quando, cosa, esito, durata
- 3 clic su una riga con esecuzione salvata → GET /hq/runs/{id} → «Rivedi nell'algoritmo»

Risultato: Storia di tutto ciò che è successo.

Aggiunto, non previsto

Mappa operazioni

- 1 GET /hq/operations
- 2 tutte le operazioni fattibili con le loro fasi, quante volte eseguite, esito, durata

Aggiunto, non previsto

Archivio bandi (nuovo)

- 1 Lista di tutti i bandi in memoria (predefiniti, dal web, caricati a mano) con contatori e avvisi
- 2 apri un bando → GET /hq/archive/{id}
- 3 Scheda consulente (GET .../consultant): «Da dove partire» in 7 passi · «Da decidere» (regole in disaccordo) · «Cosa dice il bando» (regole con fonte) · «Cosa NON dice»: per ogni regola mancante quale controllo resta spento, quali parole cercare, dove compaiono di solito, come verificare, quante volte sono citate nei documenti · requisiti da rivedere · documenti con problemi
- 4 Documenti: testo estratto con ricerca ed evidenziazione, «apri il file originale» (PDF identico byte per byte), scarica, elimina
- 5 Regole: imposta o correggi (PUT → origine HUMAN_REVIEW), elimina
- 6 Requisiti: aggiungi a mano, riclassifica tipo/tema/controlli, elimina
- 7 Azioni: «Rileggi tutto» (ricalcola le regole automatiche, mai quelle decise da una persona), Rinomina, Esporta ZIP (scheda JSON + testi + file originali), Elimina bando (i predefiniti si possono ripristinare)

Risultato: Tutti i dati scaricati sono visibili e modificabili dal manager.

Fatto come da progetto Corrisponde alla «coda di verifica per il consulente» dello schema p.3 e al Pannello di Ingestion (Modulo 8.4).

Aggiunto, non previsto Documenti originali, cancellazione, esportazione e scheda del consulente non erano nei documenti: li hai chiesti tu il 20 settembre.

Fascicoli

- 1 GET /hq/projects e /hq/bandi
- 2 per ogni progetto: timeline, esecuzioni, documenti, registrazione
- 3 per ogni bando: regole, requisiti, fonti, progetti che lo hanno usato

Aggiunto, non previsto

Documenti

- 1 GET /hq/documents
- 2 elenco di tutto ciò che è stato lavorato: bandi caricati, import, export

Aggiunto, non previsto

Database

- 1 GET /hq/db/tables → scegli una tabella → righe paginate
- 2 elimina una riga (DELETE) · svuota la tabella (serve scrivere il nome per confermare) · scarica in CSV
- 3 la tabella «anchors» (il registro firmato) è protetta: non si cancella né si modifica

Risultato: Il manager può fare ciò che vuole con i dati, tranne rompere la catena di firme.

Aggiunto, non previsto Richiesto da te.

Caricamento bandi (Pannello di Ingestion)

- 1 Scrivi codice e nome di un bando e incolli un testo
- 2 «Estrai le regole»: POST /ingestion/catalog → /ingestion/confirm → /ingestion/extract (Stadi 2 e 3)
- 3 «Demo: due letture che non concordano»: invia due «passaggi» diversi della stessa regola → il codice le confronta → finiscono nella coda
- 4 Coda di verifica: scrivi il valore corretto → «Pubblica» → POST /ingestion/review (origine HUMAN_REVIEW)
- 5 Stato: GET /ingestion/status/{id} (regole totali, da parsing, da passaggi multipli, in revisione, riviste a mano, dal catalogo curato)

Risultato: Riproduce il flusso interno dello schema p.3.

Fatto come da progetto Confronto tra passaggi fatto dal CODICE, concordanza → pubblica, disaccordo → coda, revisione umana, cache, regole mai sovrascritte.

Fatto in parte I «passaggi» non li produce un'AI: li fornisce chi chiama l'API (qui una demo). Nessun modello linguistico è collegato.

Parte B · I motori dietro le quinte

B1. Il motore del budget (POST /budget/validate)

- 1

Per ogni voce il motore crea una «riga» e stabilisce quali controlli la riguardano (personale, beni, consulenze, spese generali, sottotipi come locazione o penali).
- 2

Personale: cerca nella tabella Fonte B la coppia CCNL + livello → ore annue, oneri sociali, TFR. Se la coppia non c'è la riga è RESPINTA (nessun valore «standard» inventato).
- 3

Costo annuo = RAL + oneri + TFR (tolti i superminimi non riconosciuti, con la maggiorazione per il tempo determinato). Costo orario = costo annuo ÷ ore contrattuali.
- 4

Confronto col tetto orario del bando (#7): se lo supera l'importo è ridotto fino al tetto. Poi tempo dedicato (FTE) × durata in mesi.
- 5

Beni: natura, aliquota d'ammortamento e mesi di durata (#17-18), requisiti (nuovo, IoT, energia, DNSH), prezzo vs benchmark, installazione, leasing senza interessi, reverse charge, manutenzione, perizia sopra soglia.
- 6

Consulenze: fornitore indipendente, tariffa vs benchmark, codice ATECO, subappalti, viaggi. Spese generali e sottotipi: forfait, comunicazione, garanzie, revisione, penali, locazione, utenze, rappresentanza.
- 7

Controlli generali su ogni voce: data di ammissibilità (#46), cumulo (#47), CUP (#50), CIG (#51), metodo di pagamento (#52), IVA (#53-54), SAL (#57), cambi (#59).
- 8

Tempo dedicato cumulato per persona (#8): se la stessa persona supera il 100% su più voci le voci vengono respinte.
- 9

Limiti in % sul totale finale (#26, #31, #36, #37): poiché il totale dipende dalle riduzioni, il motore risolve l'equazione con una formula esatta e ripartisce i centesimi, senza tentativi.
- 10

Controlli sull'intero budget (#48, #49, #55, #56, #60): cumulo con crediti d'imposta, de minimis, cash-flow, variazione tra capitoli, coerenza globale.
- 11

Ogni voce riceve un'impronta SHA-256 (dati della voce, riferimenti di Fonte A/B/C e loro versioni, importi, stato) e diventa la foglia dell'albero di Merkle.
- 12

Il riassunto scritto è composto con le cifre del motore e ricontrollato dal validatore numerico (ogni numero del testo deve stare nel JSON); se non torna, si usa il testo fisso.
- 13

Punteggio = percentuale (arrotondata per difetto) di controlli superati su quelli eseguiti. Un controllo senza dato è «non valutato» e non conta come superato.
- Regola d'oro:

le regole che il bando non dà restano vuote e il controllo relativo si spegne; il motore non inventa mai un valore di partenza.

B2. I 60 controlli

Tutti e 60 sono implementati e provati (la «prova completa» a 46 voci li fa scattare tutti). Ognuno scrive nella traccia: superato, ridotto, respinto, in attesa di documento o non valutato.

Blocco	Controlli	Da cosa dipendono
Criteri 1-15 · Personale e CCNL	1 inquadramento CCNL · 2 RAL · 3 costo orario · 4 TFR · 5 oneri · 6 superminimi · 7 tetto orario del bando · 8 tempo dedicato (FTE) e somma ≤ 100% · 9 trasferte e diarie · 10 attività B2B/B2G · 11 straordinari · 12 tempo determinato · 13 collaboratori occasionali · 14 livello/complessità · 15 busta paga vs dichiarato	Sempre sui dati della voce; #7, #11, #15 si attivano solo se il bando dà la regola.
Criteri 16-30 · Beni strumentali	16 natura del bene · 17 aliquota di ammortamento · 18 pro-rata sui mesi · 19 nuovo di fabbrica · 20 interconnessione (IoT) · 21 risparmio energetico · 22 prezzo vs benchmark · 23 installazione · 24 leasing (no interessi) · 25 reverse charge · 26 beni immateriali (quota massima) · 27 manutenzione · 28 DNSH · 29 uso esclusivo · 30 perizia giurata	#19-23, #26, #28, #30 dipendono da regole del bando; gli altri lavorano sui dati.

Criteri 31-45 · Consulenze e spese generali	31 massimale % consulenze · 32 fornitore indipendente · 33 tariffa vs benchmark · 34 codice ATECO · 35 subappalti · 36 spese generali/forfait · 37 comunicazione · 38 fideiussione · 39 revisione contabile · 40 viaggi · 41 sanzioni e penali · 42 locazione · 43 utenze · 44 rappresentanza · 45 DURC	#31, #32, #34-39 dipendono da regole del bando.
Criteri 46-60 · Tempi, prezzi, cumulo	46 data di decorrenza · 47 non cumulabilità · 48 cumulo crediti d'imposta (GBER) · 49 de minimis · 50 CUP · 51 CIG · 52 metodi di pagamento · 53 IVA indetraibile · 54 IVA recuperabile · 55 cash-flow · 56 variazione tra capitoli · 57 rendicontazione per SAL · 58 durabilità · 59 cambi · 60 coerenza globale	#48, #49, #55, #56, #60 sono controlli sull'intero budget.

B3. Impronta, albero di Merkle, registro

- 1 Ogni voce ha la sua impronta SHA-256 (foglia). Prima di unirle, ogni foglia riceve un'etichetta («00») e ogni nodo un'altra («01»), così una foglia non si può scambiare con un nodo.
- 2 Le foglie si uniscono a coppie: impronta(sinistra + destra). Se una foglia resta senza compagna sale al livello successivo invariata. Si ripete fino a una sola impronta: la Merkle Root.
- 3 Il codice CEP è le prime 16 lettere della radice. Cambiando 1 € su una voce cambia tutta la radice.
- 4 «Registra l'impronta»: il progetto viene reso opaco (impronta del nome), la voce del registro contiene progetto opaco + radice + istante + hash della voce precedente e viene firmata con la chiave privata Ed25519 del server.
- 5 La catena si ricalcola a ogni controllo: se qualcuno cambia o toglie una voce passata, hash e firme non tornano e il registro diventa «alterato».
- 6 Chi verifica può fidarsi anche solo della chiave pubblica: l'attestazione si controlla offline.

Fatto in modo diverso Limite dichiarato: il registro lo tiene QUANTO, non un testimone terzo. Se il server si azzerà (Vercel) si azzerà anche il registro.

B4. Ricerca e lettura di un bando

- 1 Nome → scheda nel catalogo (solo nome). Ricerca in 4 vie (sigle di azione, cataloghi nazionali, portali per famiglia, motori di ricerca) e punteggio di pertinenza sul percorso dell'indirizzo.
- 2 Coda di download: prima ufficiali, poi PDF e atti di legge trovati nelle pagine. Massimo 16 documenti, profondità 2.
- 3 Ogni download passa dalla protezione: solo indirizzi pubblici, http/https, porte 80/443, redirect ricontrollati, limiti di peso (PDF fino a 12 MB) e di tempo, tetto di richieste.
- 4 Conversione in testo: HTML (titolo dal titolo principale), PDF (righe spezzate ricomposte), Word. Il testo si salva fino a 500.000 caratteri; il file originale resta intero.
- 5 Analisi: documenti molto lunghi ridotti ai passaggi che nominano il bando. Stadio 2 = righe «chiave: valore» e formule di prosa standard (costo orario, %, date, CUP) e regole aggiuntive. Requisiti = frasi con obblighi, divieti, limiti, numeri, classificate per tema; se ne restano pochi si tengono le frasi con numeri («Da classificare»).
- 6 Se nello stesso testo una regola ha due valori diversi (75% e 70%) non si pubblica: va in revisione. Una regola decisa da una persona non viene mai sovrascritta da una rilettura.
- 7 Per ogni documento si scrive il motivo se non ha reso nulla: testo illeggibile, troppo breve, lungo e non pertinente, solo descrittivo.

Fatto in parte Nessun modello linguistico: tutto è codice deterministico. Limiti dichiarati: pagine costruite in JavaScript, PDF scansionati (niente OCR), PDF oltre 12 MB, motori di ricerca gratuiti bloccati dai server cloud.

B5. Allocazione annuale

- 1 Variabili: per ogni spesa p e fondo j la quota coperta x_{pj} (0...copertura massima) e un «uso» binario.
- 2 Vincoli: una spesa non è coperta oltre il 100%; ogni fondo ha una dotazione massima; per categoria, quota massima sul totale coperto dal fondo; fondi non cumulabili non sulla stessa spesa; plafond de minimis residuo; finestre mensili dei fondi.
- 3 Le spese senza mese si ripartiscono su 12 mesi in centesimi interi (il resto ai primi mesi).
- 4 Obiettivo: minimizzare la spesa netta; oppure massimizzare le voci coperte; oppure minimizzare i fondi coinvolti (mantenendo almeno il 90% del risparmio massimo).
- 5 Risolutore: HiGHS tramite scipy (programmazione lineare intera). Un passo di «riparazione» riporta i residui di arrotondamento dentro i vincoli.
- 6 Risposta: piano voce per voce, uso e margine di ogni fondo, piano mensile, de minimis usato, riepilogo.

B6. Sicurezza e integrazione

Cosa	Come funziona	Stato in produzione
API per ERP	Token OAuth 2.0 (client credentials, JWT di 1 ora) oppure firma HMAC con data (tolleranza 5 minuti). Senza QUANTO_AUTH_REQUIRED l'API è aperta.	Spenta
Quartier Generale	Codice → gettone firmato di sessione; blocco dopo 5 errori; ogni azione in timeline.	Attivo, codice debole (QUANTO_1)
Webhook	Corpo firmato HMAC-SHA256 (X-Quanto-Signature); URL e segreto solo dall'ambiente, mai da richieste esterne.	Nessuno impostato
Pseudonimizzazione	Token con chiave segreta per nomi, codici fiscali, IBAN; usata sui testi delle voci scritti nella timeline.	Chiave impostata
Anti-SSRF	Ogni indirizzo scaricato è controllato e ricontrollato dopo ogni redirect.	Attivo

Parte C · Confronto con i documenti iniziali

Per ogni modulo: **come doveva essere** (dai documenti), **come l'ho sviluppato**, lo **stato** e cosa fare. Dove i due non coincidono lo dico chiaramente.

MODULO 1 · Missione

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
1.1	FKOS non scrive testi progettuali: solo componente numerica.	Nessuna funzione di scrittura. Ogni output è calcolo, allocazione, confronto o verifica.	Fatto come da progetto	

MODULO 2 · Le due missioni

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
2.1	Missione Uno: ammissibilità voce per voce, importo esatto da tabelle ufficiali (CCNL, ammortamenti, benchmark).	Fatto con 60 controlli, numeri decimali esatti. Le tabelle (Fonte B) sono valori illustrativi, non ufficiali (vedi 9.2).	Fatto in parte	Sostituire le tabelle con quelle ufficiali.
2.1	Ripartire le voci tra i pacchetti di lavoro (WP) del bando per i tetti % di categoria.	Non esistono i WP. I tetti % si applicano sul totale del progetto per categoria.	Non fatto	Decidere se serve (dipende dal bando) e aggiungere il livello WP.
2.1	Posizionare il budget rispetto ai pattern premiati (Modulo 6).	Pagina separata «Confronto», non collegata al budget della pagina Budget.	Fatto in modo da collegare	Collegare: portare il budget corrente nel Confronto.
2.1	Output esportabile in Excel, PDF o API.	Excel e PDF con CEP-ID e QR; API REST.	Fatto come da progetto	
2.2	Missione Due: l'ente carica il bilancio consuntivo (o collegamento ERP); il sistema analizza costi fissi/variabili e produce il piano dell'anno dopo con cash flow a 12 mesi.	Motore completo (12 mesi, MILP). Ma lavora su 3 spese e 3 fondi di esempio scritti nel codice. Nessun caricamento del bilancio, nessun collegamento ERP.	Fatto in parte	Pagina «Bilancio annuale»: carica, categorizza, collega ai fondi reali.
2.3	Lock-in operativo: i dati della Missione Due arrivano già puliti alla Missione Uno.	Non ci sono dati del cliente conservati (né account né organizzazioni).	Non fatto	Serve un concetto di cliente e una memoria persistente.

MODULO 4-6 · Demo, banca dati pattern, pattern matching

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
4.2 p.1	Scegli un bando di riferimento (aperto o chiuso).	La categoria è fissa: «TRANSIZIONE_5.0», e non cambia il risultato.	Non fatto	Menù dei bandi collegato ad archetipi per categoria.
4.2 p.2	Carichi un budget bozza reale o lo componi in Console.	Si scrivono a mano 4 quote %. Non si può portare il budget della pagina Budget.	Fatto in parte	Prendere le quote dal budget corrente.
4.2 p.3	Recupera i 5-10 budget storici più simili.	Non c'è una banca dati: 4 archetipi illustrativi.	Non fatto	Costruire la banca dati (5.1) e il recupero.
4.2 p.4	Pattern Similarity Score + scostamento principale con frase.	Fatto: somiglianza del coseno, archetipo più vicino, scostamento in punti %. Nota nel codice: con quote non negative il coseno è alto anche per budget diversi; conta di più lo scostamento.	Fatto come da progetto	
4.2 p.5	Simulazione di correzione in tempo reale spostando importi.	Si cambiano le quote e si preme «Confronta» (nuova chiamata): non è interattivo in tempo reale.	Fatto in parte	Ricalcolo a ogni modifica con piccolo ritardo.

4.2 p.6	Conformity Score in parallelo, con voci respinte e motivazione.	Il Conformity Score c'è, ma nella pagina Budget.	Fatto in modo di mostrare	Mostrarlo anche nel Confronto.
5.1-5.3	Banca dati da graduatorie pubbliche, dataset UE, dati del pilota; aggiornamento periodico con controllo qualità.	Nulla di questo. Solo 4 fingerprint scritti a mano e dichiarati «illustrativi» nel codice.	Non fatto	Importatori di graduatorie + controllo qualità + consenso per dati del pilota.
5.2	Fingerprint su 6 categorie (personale, beni, consulenze, generali, formazione, comunicazione).	4 categorie (personale, beni, consulenze, generali).	Fatto in parte	Aggiungere formazione e comunicazione.
6.2	Clustering periodico (k-means o gerarchico) per creare gli archetipi.	Non c'è clustering.	Non fatto	Dopo la banca dati.
6.3	Restituire archetipo, somiglianza, scostamenti voce per voce, range di punteggio storico.	Archetipo, somiglianza, medie e scostamento principale. Niente scostamenti per tutte le voci né range di punteggio.	Fatto in parte	Scostamenti per ogni categoria; range quando ci sono punteggi.
6.4	Nessun LLM nel pattern matching.	Nessun LLM.	Fatto come da progetto	

MODULO 7 · Disaccoppiamento motore / LLM (strict grounding)

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
7.1	Motore proprietario in codice puro, testabile con unit test e regression test.	Tutto il calcolo è in codice; 298 test automatici passano (rieseguiti il 21 set).	Fatto come da progetto	
7.2	L'LLM riceve solo un JSON bloccato e produce il testo con uscita strutturata.	Predisposto (interfaccia LLMClient e istruzione di sistema), ma nessun LLM è collegato: il testo è un modello fisso.	Fatto in parte	Collegare un modello (serve scelta e chiave).
7.2	Validatore numerico: ogni cifra del testo deve coincidere col JSON, altrimenti scarto e testo statico.	Fatto. Confronto esatto in Decimal, gli identificativi (LINE-001, hash) verificati a parte.	Fatto come da progetto	
7.4	L'AI dell'ingestion opera a monte, non nel calcolo.	Rispettato (non c'è AI); il punto di ingresso è il «passaggio» dello Stadio 3.	Fatto come da progetto	

MODULO 8 · Console di Budgeting

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
8.1	Ogni numero cliccabile con pannello: documento, norma, calcolo.	Si clicca la voce e l'ispettore mostra documento, tabella, regola, calcolo, controlli, impronta. I singoli numeri non sono cliccabili.	Fatto in parte	Rendere cliccabili anche i totali e i riquadri.
8.2 V1	Vista 1 Budget Canvas con l'ispettore, totale ammissibile ed escluso, Conformity.	Pagina Budget.	Fatto come da progetto	
8.2 V2	Vista 2 Allocazione: barra mensile, clic sul mese con dettaglio, simulazione «sposta questa voce sul Fondo Y».	Grafico mese per mese e flusso; what-if solo togliendo un fondo. Niente clic sul mese né spostamento di una voce.	Fatto in parte	Dettaglio mese e spostamento voce.
8.2 V3	Vista 3 Demo pattern.	Pagina Confronto (vedi sopra).	Fatto in parte	Vedi Moduli 4-6.
8.2 V4	Vista 4 Report: Excel/PDF del budget o del piano di allocazione, con note esplicative.	Excel/PDF solo del budget. Nessun export dell'allocazione. Il riassunto scritto non entra nei file.	Fatto in parte	Export allocazione + note nei file.
8.2	Barra laterale: Bandi Attivi, Bilancio Annuale, Allocazione Risorse, Demo Pattern, Report.	Barra in alto con 7 pagine; manca «Bilancio Annuale»; il Report è dentro Budget.	Fatto in modo di seguire la pagina	Segue la pagina Bilancio annuale.
8.3	Il frontend non calcola mai; ogni interazione = nuova chiamata.	Rispettato.	Fatto come da progetto	

8.4 / p.8	Vista 5: Pannello di Ingestion (uso interno) con stato pipeline e coda di verifica.	Quartier Generale: «Caricamento bandi» e «Archivio bandi».	Fatto come da progetto	
-----------	---	--	------------------------	--

MODULO 9.1 · Fonte A: normativa e regole (versione finale)

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
Stadio 1	Agente AI che scansiona in continuo tutte le fonti istituzionali (Gazzetta Ufficiale, portali ministeriali e regionali) e popola il Catalogo Bandi (solo metadati).	Nessun agente continuo e nessuna AI. La ricerca parte quando l'utente la chiede. Fonti usate: catalogo nazionale (5.889 misure), Invitalia, portali per famiglia, mappa di 5 sigle Erasmus+, motori di ricerca se c'è la chiave. Il catalogo sta in memoria 6 ore, non in database. Gazzetta Ufficiale non scansionata.	Fatto in parte	Job periodico che aggiorna un catalogo in database; Gazzetta Ufficiale; chiave di un motore di ricerca.
Trigger	Il cliente cerca il bando (nome esatto o approssimativo) nella Console.	Cerca sul web (con confronto approssimativo sul catalogo nazionale). Non cerca nell'elenco interno dei bandi già in memoria.	Fatto in parte	Ricerca per nome nell'elenco interno, poi web.
Trigger	Bando non nel catalogo → ricerca mirata immediata.	È il comportamento normale del pannello.	Fatto come da progetto	
Trigger	Il cliente conferma «è il bando giusto?» prima dell'estrazione.	Non c'è la conferma: scarica e legge subito.	Non fatto	Passo di conferma prima di scaricare/analizzare.
Cache	Se le regole del bando sono già state estratte per un altro cliente, si riusano senza costi.	Le regole pubblicate non si sovrascrivono e c'è un contatore di richieste, ma il pannello utente non controlla la cache e ricarica.	Fatto in parte	Controllo cache prima di ricaricare.
Stadio 2	Parser deterministici per tabelle standard; libreria di template che cresce con l'uso.	Parser di righe «chiave: valore» e di formule in prosa (costo orario, %, date, CUP), più una tassonomia di requisiti in italiano e inglese. Non riconosce tabelle dei PDF come tali e non ha una libreria di template.	Fatto in parte	Parser di tabelle e libreria di template per i formati ministeriali.
Stadio 3	N estrazioni AI indipendenti della stessa regola; il codice le confronta; concordanza → pubblica; disaccordo → coda del consulente.	Il confronto è fatto dal codice e funziona (test). Manca chi produce i passaggi: nessuna AI collegata. Nel flusso utente si usa l'estrazione deterministica e, se in uno stesso testo compaiono valori diversi (es. 75% e 70%), la regola va in revisione.	Fatto in parte	Collegare estrattori AI (scelta del modello e chiave).
Coda	Coda di verifica solo sui reali disaccordi; il consulente corregge; regola pubblicata con riferimento e passaggi tracciati.	Fatto: coda, pubblicazione con origine HUMAN_REVIEW, tracciamento; scheda del consulente con cosa dice e non dice.	Fatto come da progetto	
Audit	Audit di calibrazione continua: campione casuale anche tra le regole già pubblicate, revisore senior, ricalibrazione.	Non c'è.	Non fatto	Estrazione periodica di un campione e strumento di revisione.
Reattivo	Il cliente segnala un dubbio → ticket al Reparto di Consulenza → ricalibrazione.	Non c'è (dipende dalla Fase 4).	Fase futura nei documenti	
Indice	Regole e testi divisi in unità (chunking), trasformati in vettori (embedding) e indicizzati in un database vettoriale.	Non c'è. In sostituzione: ricerca testuale semplice nei documenti (solo per il manager).	Non fatto	Database vettoriale + ricerca semantica.

MODULO 9.2 · Fonte B: tabelle di mercato

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
------	--------------------	----------------------	-------	---------------------

9.2	Un connettore per fonte (Ministero del Lavoro, ISTAT, portali di categoria) scarica periodicamente e converte in matrici numeriche.	Nessun connettore.	Non fatto	Connettori o caricamento guidato dei dati ufficiali.
9.2	Database relazionale, tabelle versionate per data di validità (le vecchie restano per calcoli retroattivi).	Una tabella scritta nel codice con 7 combinazioni CCNL/livello (Terzo Settore 1-2-3, Metalmeccanica 3-5, Commercio 3-4). Ha una versione («2026.1-ILLUSTRATIVE») che entra nell'impronta della riga, quindi un ricalcolo con tabelle diverse cambia la radice. Non è in database né per data di validità.	Fatto in parte	Tabelle nel database con data di validità.
9.2	Valori ufficiali (ore annue, oneri, TFR, ammortamenti, benchmark).	Valori ILLUSTRATIVI: dichiarati così nel codice e in questo documento. Non sono le tabelle ufficiali.	Non fatto	PRIORITÀ ALTA: senza tabelle ufficiali il calcolo non vale per un cliente reale.

MODULO 9.3 e 16 · Fonte C: dati del cliente e privacy

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
9.3	Parser proprietario con OCR per buste paga, F24, bilanci, piano dei conti; estrae importi, livelli, date, codici.	Nessun parser di documenti contabili e nessun OCR. Si inserisce a mano, si importa Excel/CSV con un template, o si usano scenari di prova.	Non fatto	PRIORITÀ ALTA per la Missione Due e per i dati del personale.
9.3 / 15.3	Se la lettura non è abbastanza sicura, il campo è segnalato per verifica manuale (fail-safe).	Non c'è lettura. Il principio equivalente c'è: se un dato manca il controllo risulta «non valutato», mai «superato».	Fatto in modo diverso	Rivedere quando ci sarà il parser.
16.1	Prima di elaborare, nomi, codici fiscali, IBAN diventano token; il documento si ricompone coi dati veri solo sul lato client.	Il modulo di pseudonimizzazione esiste (token con chiave segreta, non un semplice hash) e ripulisce IBAN e codici fiscali dai testi delle voci scritte nella timeline. Non è collegato a un flusso di documenti perché non se ne caricano. La ricomposizione lato client non c'è.	Fatto in parte	Collegarlo al caricamento dei documenti; ricomposizione lato client.
16.2	Pseudonimizzazione sul server in cloud isolato e cifrato, con contratto di trattamento dati (DPA).	La chiave di pseudonimizzazione è impostata in produzione. Non c'è alcun DPA (è un atto legale) e non ho verificato la cifratura del database.	Fatto in parte	DPA e verifica cifratura.

MODULO 10 · Da costo grezzo a riga validata

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
10 p.1-8	Input → RAL (Fonte C) → ore, oneri, TFR (Fonte B) → costo orario = (RAL + oneri + TFR) ÷ ore → tetto del bando (Fonte A) → quota FTE e durata → criteri 1-15 → riga VALIDATA bloccata con identificativo e riferimenti, oppure RESPINTA con motivo.	Fatto. Con l'esempio del documento (PM Junior, 50%, 38.000 €) il codice dà 31,74 €/ora e 26.282,70 € annuali, stato «Ammessa». Riga bloccata con impronta SHA-256 che comprende riferimento documento, versione tabella e versione regole.	Fatto come da progetto	Attenzione: il documento riporta 31,58 €/ora e 26.148 €. Con i suoi stessi numeri (38.000 + 11.400 + 3.167 = 52.567 ÷ 1.656) fa 31,74: l'esempio nel documento ha un errore di calcolo. Il codice è giusto.
10 p.2	La RAL si legge dalla busta paga caricata.	La RAL si inserisce a mano (il controllo #15 la confronta con la busta paga se indicata).	Fatto in parte	Vedi Fonte C.
10 p.9	L'LLM genera solo la frase; il numero è iniettato dal motore.	Frase da modello fisso con cifre iniettate dal motore e controllate dal validatore.	Fatto in modo diverso	Vedi 7.2.

MODULO 11 · I 60 criteri

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
11	60 criteri in 4 blocchi, ognuno una funzione autonoma e testabile.	Tutti e 60 sono implementati: la «prova completa» a 46 voci li fa scattare tutti. Non sono 60 funzioni separate ma controlli raggruppati dentro il motore per categoria (personale, beni, consulenze, spese generali, sottotipi, generali, budget). Sono provati da 404 righe di test dedicati.	Fatto in modo diverso	Nessun intervento: funziona; sarebbe solo una scelta di organizzazione del codice.
11	Non è previsto cosa fare se manca un dato.	Il controllo non parte e risulta «non valutato», mai «superato». Le regole non definite nel bando restano vuote: nessun valore inventato.	Aggiunto, non previsto	

MODULO 12 · Allocazione annuale

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
12.2	Ottimizzazione vincolata: x_{ij} quota della voce i coperta dal fondo j ; vincoli: dotazione del fondo, tetti % per categoria, non cumulabilità (#47), de minimis (#49).	Fatto come programma lineare intero (HiGHS). In più: finestre mensili di attività dei fondi e riparto in centesimi interi.	Fatto come da progetto	
12.2	3 funzioni obiettivo.	Fatte: minimizzare spesa netta, massimizzare voci coperte, minimizzare fondi coinvolti.	Fatto come da progetto	
12.3 p.1	Ingestion del bilancio storico (Fonte C) e categorizzazione automatica.	Non c'è.	Non fatto	Vedi Fonte C.
12.3 p.2	Interrogazione della Fonte A per i fondi attivi compatibili con ciascuna categoria.	Non c'è: i fondi sono scritti nel codice.	Non fatto	Ricavare i fondi dai bandi in memoria.
12.3 p.3-5	Matrice di compatibilità, solutore standard, piano con margine di sicurezza.	Fatto.	Fatto come da progetto	
12.3 p.6	What-if: escludi una fonte, ricalcolo istantaneo.	Fatto.	Fatto come da progetto	
12.3	Riepilogo dell'LLM.	Riepilogo da modello fisso.	Fatto in modo diverso	Vedi 7.2.

MODULO 13 e 14 · Dove interviene l'AI · Metriche

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
13	Tabella: l'LLM solo per ricerca documenti, lettura di prosa libera e testo di spiegazione.	Nessun LLM in tutto il codice. Ricerca, lettura e testi sono fatti con codice deterministico o lasciati a un punto di estensione. Vantaggio: nessuna allucinazione e nessun costo per chiamata; svantaggio: meno copertura sulla prosa libera e sulla scoperta di documenti.	Fatto in modo diverso	Decisione tua: se collegare un modello e con quali limiti.
14.1	Conformity Score misurabile con casi di test noti.	Punteggio in app; 298 test automatici.	Fatto come da progetto	
14.2	Pattern Similarity Score, dichiarato come indicazione, non previsione.	Fatto e dichiarato nell'app.	Fatto come da progetto	
14.3	Tasso di successo reale: non compare finché non c'è storico.	Non compare.	Fatto come da progetto	

MODULO 15 · Stack e infrastruttura

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
15.1	Due servizi: orchestrazione e calcolo.	Un solo servizio (FastAPI) con moduli separati.	Fatto in modo di	Nessuno per ora.
15.1	Numeri decimali a precisione fissa.	Fatto (Decimal in tutto il motore).	Fatto come da progetto	
15.2	PostgreSQL + database vettoriale.	SQLite. Su Vercel il file sta in cartella temporanea: si azzerava a ogni riavvio del server. Documenti scaricati, bandi cercati ed esecuzioni si perdono.	Non fatto	PRIORITÀ ALTA: database esterno (serve tuo consenso: account terzo, es. Neon).
15.3	OCR open-source per Fonte C.	Non c'è.	Non fatto	Vedi Fonte C.
15.5	Hosting in UE, servizi containerizzati, test di regressione a ogni modifica.	Funzione Vercel a Francoforte. Nessun container. Test automatici su GitHub (GitHub Actions: 298 test backend + build frontend) a ogni push.	Fatto in parte	Container e database UE persistente.

MODULO 17 · Registro crittografico (CEP)

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
17.2	SHA-256 per ogni pezza d'appoggio (busta paga, fattura).	Non si caricano i file di appoggio: l'impronta riguarda i dati della voce e i riferimenti. Per i documenti dei BANDI invece ogni file scaricato ha la sua impronta SHA-256.	Non fatto	Dopo il caricamento dei documenti di Fonte C.
17.2	Triplo hash per riga: documento, regola con versione, calcolo.	Un'unica impronta per riga che include riferimento al documento, versione tabella, versione regole, regole applicate, importi e stato.	Fatto in modo di	Decidere se separare in tre impronte.
17.2	Albero di Merkle e Merkle Root unica.	Fatto, con etichette diverse per foglie e nodi (contro attacchi di confusione), impronta senza compagno che «sale» invariata, prova di una singola riga.	Fatto come da progetto	
17.3 L1	PDF/XLSX con CEP-ID stampato.	Fatto.	Fatto come da progetto	
17.3 L2	QR code e URL all'Auditor Portal; ricalcolo dell'albero.	Fatto.	Fatto come da progetto	
17.3	Report di Asseverazione Crittografica in 45 secondi.	Verdetto a schermo; nessun documento scaricabile; tempi non misurati.	Fatto in parte	Generare il report in PDF.
17.1/4	Il CEP è Fase 2, si costruisce dopo il pilota.	Costruito subito. Al posto dell'ancoraggio blockchain del racconto iniziale: registro con catena di hash e firma Ed25519, attestazione verificabile offline, chiave pubblica esposta.	Fatto in modo di	Limite dichiarato: il registro lo tiene QUANTO, non un testimone terzo; per rafforzarlo si può pubblicare periodicamente l'ultimo hash.

MODULO 18, 19 · Assicurazione e Reparto di consulenza

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
18	Componente assicurativa (Fase 3): master binder, garanzia, rischio correlato.	Non costruita, come previsto dai documenti.	Fase futura nei documenti	
19	Reparto di consulenza (Fase 4): segnalazione automatica dei casi limite, rete di consulenti, commissione.	Non costruito. Esiste solo la scheda del consulente nel Quartier Generale, che è uno strumento interno di lavoro, non il Reparto.	Fase futura nei documenti	

MODULO 20-22 · Go-to-market, prezzi, concorrenza

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
21.3	Abbonamento base, crediti a consumo, success fee, commissione Fase 4.	Nulla: non esistono account, organizzazioni, abbonamenti, crediti né fatturazione.	Non fatto	Da progettare quando parte il pilota (Modulo 23 del vecchio dossier: 3-5 clienti).
20-22	Segmenti, canali, partnership, difesa competitiva.	Contenuti di business, non di codice.	Fase futura nei documenti	
2.2 / 24	Integrazione con ERP (Zucchetti, TeamSystem).	Esiste l'API con OAuth 2.0 e firma HMAC, ma è spenta finché non si impostano QUANTO_AUTH_REQUIRED e i segreti. Nessun connettore verso un ERP.	Fatto in parte	Attivare l'autenticazione prima di aprire ai clienti.

SCHEMA API · Endpoint e notifiche

Rif.	Come doveva essere	Come l'ho sviluppato	Stato	Da sistemare / note
API 1-4	POST /budget/validate, POST /allocation/optimize, POST /pattern/match, GET /ingestion/status/{bando}.	Tutti e 4 esistono con quel percorso. Cambiano i nomi dei campi: per esempio grant_rules invece di grant_context + rule_version_hash; historical_expenses inline invece di historical_balance_ref; total_gross_expense_eur invece di total_cost_eur. Lo stato dell'ingestion ha campi in più (da passaggi multipli, riviste a mano, dal catalogo curato).	Fatto in modo di	Decidere se allineare il codice allo schema o l'inverso; è una scelta, non un errore.
API 5	7 notifiche webhook: budget.validated, allocation.optimized, criteria.failed, pattern.matched, consultation.requested, rule.extracted_low_confidence, bando.coverage_activated.	Attive: budget.validated, allocation.optimized, criteria.failed, pattern.matched, bando.coverage_activated. Al posto di rule.extracted_low_confidence (che era legata al vecchio «punteggio di confidenza») c'è rule.disagreement_detected, coerente col nuovo confronto tra passaggi. Aggiunta budget.registered. Non c'è consultation.requested (Fase 4). Si inviano solo se sono impostati QUANTO_WEBHOOK_URLS e il segreto; firmate HMAC-SHA256.	Fatto in modo di	Nessuno per ora.
API	OAuth 2.0 e firma HMAC per l'integrazione ERP.	Fatto (token JWT firmato a 1 ora; firma della richiesta con data per impedire il riuso). In produzione non è attivo.	Fatto in parte	Impostare QUANTO_AUTH_REQUIRED e i segreti.

Parte D · Cosa sistemare, in ordine di importanza

Dimensione stimata: **S** = pochi giorni · **M** = una-due settimane · **L** = più di due settimane. Sono stime di massima, non impegni. Dove serve una tua decisione, un servizio esterno o dati che non ho, lo scrivo.

ALTA Cose che rendono i numeri veri o non li fanno perdere

Cosa	Com'è ora	Cosa fare	Peso
Memoria persistente	Su Vercel il database (SQLite) si azzerà: bandi cercati, documenti scaricati, esecuzioni e registro vanno persi a ogni riavvio del server.	Database esterno (PostgreSQL). Serve il tuo consenso perché è un servizio terzo.	M
Tabelle ufficiali (Fonte B)	Le tabelle di CCNL, oneri, TFR, ammortamento e benchmark sono illustrative e coprono 7 combinazioni.	Caricare le tabelle ufficiali versionate per data di validità e (poi) i connettori. Servono i dati e un consulente che li validi.	L
Documenti del cliente (Fonte C)	Nessun parser di buste paga, bilanci, F24 e nessun OCR; nessun controllo di «confidenza di lettura».	Caricamento + lettura + segnalazione dei campi incerti, collegato alla pseudonimizzazione.	L
Missione Due con dati veri	L'allocazione usa 3 spese e 3 fondi di esempio.	Pagina «Bilancio annuale» (carica, categorizza) e fondi ricavati dai bandi in memoria.	L
Flusso di ricerca come da schema	Manca la ricerca nell'elenco interno, la conferma «è il bando giusto?» e il controllo della cache.	Ricerca interna → conferma → cache → estrazione → (web solo se manca).	M
Accesso e sicurezza	API aperta (autenticazione spenta), codice manager QUANTO_1, sito dietro la protezione Vercel.	Impostare QUANTO_AUTH_REQUIRED, QUANTO_HQ_CODE forte, e decidere se togliere la Deployment Protection.	S

MEDIA Completare i motori descritti

Cosa	Com'è ora	Cosa fare	Peso
Confronto (pattern)	Categoria fissa, 4 quote, archetipi illustrativi, niente banca dati né clustering, niente simulazione live.	Scelta del bando, quote dal budget corrente, 6 categorie, ricalcolo live, Conformity Score accanto; poi banca dati e k-means.	M-L
Al dove i documenti la prevedono	Nessun LLM: Stadio 1 (agente), Stadio 3 (N passaggi), Renderer del testo.	Collegare un modello ai punti di estensione già pronti (ai_passes e LLMClient). Serve scelta del modello e chiave.	M
Catalogo continuo (Stadio 1)	La ricerca parte a richiesta e il catalogo è in memoria per 6 ore.	Job periodico che aggiorna un catalogo in database, Gazzetta Ufficiale inclusa.	M
Console	Numeri cliccabili; clic sul mese e spostamento voce in Allocazione; export dell'allocazione; note nei file Excel/PDF.	Completare le viste 2 e 4 come nel Modulo 8.	M
Report di asseverazione	Solo verdetto a schermo.	PDF scaricabile e misura dei tempi.	S
Forma dell'API	Nomi dei campi diversi dallo schema.	Decidere se allineare.	S

BASSA Rete di sicurezza e rifiniture

Cosa	Com'è ora	Cosa fare	Peso
Audit di calibrazione e segnalazioni	Non ci sono campionamento casuale né ticket.	Strumento di campionamento e coda di segnalazioni.	M
Ricerca semantica	Nessun chunking né vettori.	Database vettoriale sulle regole e sui testi.	M
Pacchetti di lavoro (WP)	Assenti.	Solo se un bando lo richiede.	M

Copertura dei bandi	Mappa delle sigle: solo 5 codici Erasmus+; niente OCR; niente PDF sopra 12 MB; niente pagine costruite in JavaScript.	Aggiungere sigle e programmi, OCR, lettura a blocchi.	M
Contenitori e servizi separati	Un solo servizio, niente container.	Solo quando serve scalare.	M

FUTURA Fasi che i documenti stessi rimandano

Cosa	Com'è ora	Cosa fare	Peso
Fasi 3 e 4	Componente assicurativa e Reparto di consulenza.	Da avviare dopo un pilota con clienti reali.	—
Clienti, prezzi, fatturazione	Non esistono account né piani (abbonamento, crediti, success fee).	Da progettare per il pilota.	—
Integrazione ERP	Nessun connettore (Zucchetti, TeamSystem).	Dopo l'attivazione dell'autenticazione.	—

Dove serve una tua decisione prima che io parta: (1) database esterno e provider; (2) se collegare un modello linguistico, quale e con quali limiti; (3) chi fornisce e valida le tabelle ufficiali; (4) se tenere la forma dell'API attuale o quella dello schema; (5) se togliere la protezione Vercel per aprire il sito agli utenti.

Parte E · Cose aggiunte che non erano nei documenti

Cosa	Che cos'è
Ricerca e download vero dei documenti	Scarica pagine, PDF e Word, conserva il file originale e il testo, salva le impronte, spiega perché un documento non ha reso nulla. Protezione contro l'uso improprio del server (SSRF).
Scoperta senza motori di ricerca	Catalogo nazionale incentivi.gov.it (5.889 misure), Invitalia, portali per famiglia, mappa delle sigle Erasmus+. Necessario perché i motori gratuiti sono bloccati dai server cloud.
Lettura di requisiti in italiano e inglese	Tassonomia di temi (agevolazione, importi, età, territori, scadenze...) e tipi (obbligo, divieto, limite, informazione), riordino dei PDF spezzati, riconoscimento di testi illeggibili.
Vista Algoritmo	Riproduzione della traccia reale, mappa 60 controlli × voci, cascata degli importi, albero di Merkle.
Laboratorio Merkle e Guida	Spiegazione passo passo, glossario di oltre 40 termini, esempi per ogni funzione, aiuti «?» in ogni pagina.
Quartier Generale con Archivio bandi	Timeline di ogni operazione, mappa delle operazioni, fascicoli, documenti, database con eliminazione ed esportazione (registro firmato protetto), scheda del consulente, esportazione ZIP.
Registro con firma Ed25519	Al posto della blockchain: catena di hash append-only con firma, attestazione verificabile offline.
Punto di controllo dei dati mancanti	Ogni controllo senza dato risulta «non valutato», la griglia dei controlli attivi per bando dice cosa resta spento.
Nuova grafica	Bianco, vetro liquido, giallo del marchio, font Now (in attesa dei file; nel frattempo Outfit), icone, loghi.
CI e test	GitHub Actions con 298 test automatici e build del frontend a ogni push.

Parte F · Limiti di questa verifica

Per non farti credere più di quello che so:

- **Letti per intero:** il Dossier v2.2 e lo schema dei processi del 17 settembre. **FKOS_v2_completo** l'ho confrontato riga per riga con la v2.2: cambia solo l'ingestion (Moduli 7.4, 8.4, 9.1, 13, 15.4), che ho letto per intero.
- **Letti solo in parte:** Dossier v2.1, Dossier PROGETTOBUDGET (struttura e Modulo 23) e raccontoBUDGET (struttura). Sono versioni precedenti degli stessi contenuti; le ho confrontate con la v2.2 riga per riga solo dove differiscono.
- **Non confrontato:** il piano economico (.xlsx): è un documento di business; ne ho solo verificato la struttura (6 fogli).
- **Codice:** ho letto i moduli principali (motore, ingestion, allocazione, pattern, registro, esportazioni, autenticazione, webhook, anonimizzatore), tutte le rotte dell'API e la lista dei componenti dell'interfaccia. Non ho riletto riga per riga

le 868 righe del motore.

- **Prove:** ho rieseguito i 298 test (tutti superati) e ho rifatto girare l'esempio numerico del documento. Ho controllato la prova completa a 46 voci: fa scattare tutti i controlli. La grafica l'ho controllata a schermo in locale.
- **Stime dei tempi (S/M/L)** sono mie e vanno prese come ordine di grandezza.
- **Nulla qui vale come parere legale o fiscale.** Le regole dei bandi vanno sempre verificate sul testo ufficiale.

Appendice A · Tutte le funzioni dell'API

Elenco generato direttamente dal codice: ogni riga è una funzione che il server offre. «HQ» = serve il gettone del manager.
«token» = protetta dall'autenticazione ERP quando è attiva.

Biblioteca dei bandi e ricerca (9)

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/bandi/research/search	Cerca il bando sul web: pagine e documenti, ufficiali per primi	token
POST	/api/v2/bandi/research/fetch	Scarica una pagina o un PDF, ne salva il testo in memoria e restituisce i link utili	token
POST	/api/v2/bandi/research/analyze	Legge tutti i documenti in memoria: regole, requisiti, dati chiave, riferimenti di legge	token
GET	/api/v2/bandi/{bando_id}/sources/{sha256}	Testo integrale di una fonte salvata in memoria (solo Quartier Generale)	
GET	/api/v2/bandi	Elenco dei bandi con copertura dei 60 criteri	
GET	/api/v2/bandi/references	Quadri normativi di riferimento (es. de minimis)	
GET	/api/v2/bandi/{bando_id}	Dettaglio del bando: regole con fonte, requisiti, copertura dei criteri, lacune	
POST	/api/v2/bandi/{bando_id}/select	Seleziona il bando: restituisce le regole pronte per la validazione	
POST	/api/v2/bandi/upload	Carica il testo o il PDF di un bando: estrae regole, ambito e requisiti	

Missione Uno · Budget (8)

Protetti dall'autenticazione ERP solo se QUANTO_AUTH_REQUIRED=1.

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/budget/validate	Valida deterministicamente un budget di progetto (Missione Uno)	
POST	/api/v2/budget/export/xlsx	Esporta il budget validato in XLSX con CEP-ID e QR (Layer 1)	
POST	/api/v2/budget/export/pdf	Esporta il budget validato in PDF con CEP-ID e QR (Layer 1)	
GET	/api/v2/budget/fields	Catalogo dei campi di una voce di costo (per l'editor e il template)	
GET	/api/v2/budget/template.xlsx	Template Excel con tutte le colonne, esempi e guida ai campi	
POST	/api/v2/budget/import	Importa le voci da un file .xlsx o .csv (errori riga per riga)	
GET	/api/v2/budget/demo	Scenario demo adattato alle regole del bando (stress-test 46 voci o progetto realistico)	
GET	/api/v2/budget/criteria	I 60 criteri del Deterministic Engine	

Confronto (pattern) (1)

Come sopra.

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/pattern/match	Confronta un budget bozza con gli archetipi vincenti	

Missione Due · Allocazione (1)

Come sopra.

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/allocation/optimize	Mappa le spese correnti sulle linee di finanziamento attive (Missione Due)	

Registro e Auditor Portal (8)

«register» è protetto se l'autenticazione è attiva; le verifiche dell'Auditor Portal sono pubbliche.

Metodo	Percorso	Cosa fa	Accesso
GET	/api/v2/registry/status	Integrità della catena del registro e chiave di firma attiva	
POST	/api/v2/registry/merkle-lab	Didattica: calcola la Merkle Root passo passo su righe di esempio (stesso algoritmo del sistema)	
GET	/api/v2/registry/public-key	Chiave pubblica Ed25519 di firma (radice di fiducia: da fissare fuori banda)	
POST	/api/v2/registry/register	Registra la Merkle Root di un budget nel registro firmato	token
GET	/api/v2/registry/attestation/{project_id}	Attestazione firmata di un progetto registrato	
GET	/api/v2/registry/verify/{project_id}	Verifica una Merkle Root contro il registro (Auditor Portal)	
POST	/api/v2/registry/verify/recompute	Ricalcola la Merkle Root dai dati originali e la verifica	
POST	/api/v2/registry/verify/attestation	Verifica OFFLINE di un'attestazione con la sola chiave pubblica di fiducia	

Ingestion della Fonte A (8)

Come sopra.

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/ingestion/catalog	Stadio 1: censisce un bando nel catalogo (solo metadati)	
GET	/api/v2/ingestion/catalog	Catalogo dei bandi	
POST	/api/v2/ingestion/confirm/{bando_id}	Il cliente conferma il bando: trigger dell'estrazione (o cache hit)	
POST	/api/v2/ingestion/extract	Stadi 2 e 3: estrazione deterministica + riconciliazione dei passaggi multipli	
GET	/api/v2/ingestion/status/{bando_id}	Stato dell'ingestion normativa per un bando (Pannello Interno)	
GET	/api/v2/ingestion/review-queue	Coda di verifica umana: solo i casi di reale disaccordo tra i passaggi	
POST	/api/v2/ingestion/review	Il consulente risolve un disaccordo: la regola viene pubblicata	
GET	/api/v2/ingestion/grant-rules/{bando_id}	GrantRuleSet composto dalle sole regole pubblicate (pronto per /budget/validate)	

Autenticazione ERP (1)

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/auth/token	OAuth 2.0 client-credentials: restituisce un bearer token (1 ora)	

Quartier Generale (30)

Tutti richiedono il gettone del manager (tranne il login).

Metodo	Percorso	Cosa fa	Accesso
POST	/api/v2/hq/login	Accesso al Quartier Generale con il codice manager	
GET	/api/v2/hq/overview	Panoramica: contatori, storage, registro, operazioni, ultimi eventi	HQ
GET	/api/v2/hq/operations	Mappa di tutte le operazioni fattibili con le loro fasi e le statistiche	HQ
GET	/api/v2/hq/timeline	Timeline degli eventi (filtrabile)	HQ
GET	/api/v2/hq/runs/{run_id}	Esecuzione salvata (richiesta e risposta complete, con la traccia dell'algoritmo)	HQ

GET	/api/v2/hq/projects	Elenco dei progetti lavorati	HQ
GET	/api/v2/hq/projects/{project_id}	Fascicolo di un progetto: timeline, esecuzioni, documenti, registrazione	HQ
GET	/api/v2/hq/bandi/{bando_id}	Fascicolo di un bando: regole, requisiti, fonti, progetti che lo hanno usato	HQ
GET	/api/v2/hq/documents	Documenti lavorati (bandi caricati, import, export)	HQ
GET	/api/v2/hq/db/tables	Tabelle del database (sola lettura)	HQ
GET	/api/v2/hq/db/table/{table}	Righe di una tabella (sola lettura, paginate)	HQ
GET	/api/v2/hq/archive	Tutti i bandi in memoria (predefiniti, dal web, caricati) con contatori	HQ
GET	/api/v2/hq/archive/{bando_id}	Tutto di un bando: regole (anche in disaccordo), requisiti, fonti e file originali	HQ
GET	/api/v2/hq/archive/{bando_id}/consultant	Scheda per il consulente: cosa dicono i documenti, cosa non dicono, cosa cercare e come verificare	HQ
GET	/api/v2/hq/archive/{bando_id}/sources/{sha256}/text	Testo estratto di un documento	HQ
GET	/api/v2/hq/archive/{bando_id}/sources/{sha256}/file	File originale del documento (PDF, pagina, Word) così come è stato scaricato	HQ
DELETE	/api/v2/hq/archive/{bando_id}/sources/{sha256}	Elimina un documento (testo e file originale) dalla memoria del bando	HQ
POST	/api/v2/hq/archive/{bando_id}/reanalyze	Rilegge tutti i documenti in memoria (ricalcola regole automatiche e requisiti)	HQ
PUT	/api/v2/hq/archive/{bando_id}/rules/{key}	Imposta o corregge una regola (decisione del manager)	HQ
DELETE	/api/v2/hq/archive/{bando_id}/rules/{key}	Elimina una regola	HQ
POST	/api/v2/hq/archive/{bando_id}/requirements	Aggiunge un requisito a mano	HQ
PATCH	/api/v2/hq/archive/{bando_id}/requirements/{seq}	Riclassifica un requisito (tipo, tema, controlli collegati)	HQ
DELETE	/api/v2/hq/archive/{bando_id}/requirements/{seq}	Elimina un requisito	HQ
PATCH	/api/v2/hq/archive/{bando_id}	Rinomina il bando	HQ
DELETE	/api/v2/hq/archive/{bando_id}	Elimina il bando con tutti i suoi documenti, file, regole e requisiti	HQ
POST	/api/v2/hq/archive/restore-defaults	Ripristina i bandi predefiniti eliminati	HQ
GET	/api/v2/hq/archive/{bando_id}/export.zip	Scarica tutto il bando: scheda JSON, testi e file originali	HQ
DELETE	/api/v2/hq/db/table/{table}/row/{rowid}	Elimina una riga (tranne il registro firmato)	HQ
DELETE	/api/v2/hq/db/table/{table}	Svuota una tabella (serve confirm= <nome tabella>); il registro firmato è escluso	HQ
GET	/api/v2/hq/db/table/{table}/export.csv	Esporta una tabella in CSV (testi completi, file come dimensione)	HQ

Totale: 66 funzioni.

Appendice B · Variabili di configurazione

Sono impostazioni del server. Nella colonna a destra, lo stato su Vercel (produzione) al 21 settembre.

Nome	A cosa serve	In produzione
QUANTO_AUTH_REQUIRED	Accende la protezione dell'API (OAuth 2.0 / HMAC).	NON impostata

QUANTO_JWT_SECRET · QUANTO_OAUTH_CLIENTS · QUANTO_HMAC_SECRET	Segreti per token, client e firma delle richieste.	NON impostate
QUANTO_HQ_CODE · QUANTO_HQ_SECRET	Codice del Quartier Generale e chiave del suo gettone.	NON impostate (vale QUANTO_1)
QUANTO_SIGNING_KEY	Chiave Ed25519 di firma del registro.	Impostata
QUANTO_TRUSTED_PUBLIC_KEYS	Chiavi pubbliche di fiducia (rotazione).	NON impostata
QUANTO_PII_KEY	Chiave di pseudonimizzazione.	Impostata
QUANTO_PUBLIC_URL	Indirizzo usato nel QR dei file esportati.	Impostata
QUANTO_DB_PATH	Dove sta il database.	NON impostata (cartella temporanea: si azzerà)
QUANTO_WEBHOOK_URLS · QUANTO_WEBHOOK_SECRET	Dove e come inviare le notifiche.	NON impostate (nessuna notifica)
QUANTO_BRAVE_API_KEY · QUANTO_SERPER_API_KEY · QUANTO_TAVILY_API_KEY	Chiavi dei motori di ricerca a pagamento/gratuiti.	NON impostate
QUANTO_CORS_ORIGINS	Siti autorizzati a chiamare l'API.	NON impostata